

Routing Algorithms

Routing algorithms are designed to determine efficient paths (routes) from senders to receivers across a network of routers. The primary goal is usually to find the path with the **least cost**, though in practice other concerns such as **policy restrictions** also affect routing (e.g., a router from one organization might refuse to forward packets from another organization). Regardless of whether the network uses a **per-router control** or a **centralized control** system, routing always requires a clear sequence of routers that a packet must pass through. Thus, routing algorithms are a **fundamental networking concept**.

To represent routing problems, computer networks are modeled as **graphs**.

- A graph $G=(N,E)$ consists of:
 - N : a set of **nodes** (representing routers).
 - E : a set of **edges** (representing physical links between routers).
- Example: In BGP (Border Gateway Protocol), nodes can represent entire **networks**, while edges represent **peering relationships** between networks.

Each edge in the graph has a **cost** value. This cost can represent:

- Physical length of a link (e.g., transoceanic links cost more than short terrestrial ones).
- Link speed.
- Monetary costs of using the link.

Formally, the cost of an edge (x,y) is denoted as $c(x,y)$. If there is no direct edge, then $c(x,y)=\infty$. In undirected graphs, which we focus on, $c(x,y)=c(y,x)$. If $(x,y) \in E$, then node y is called a **neighbor** of node x .

The **goal of a routing algorithm** is to find the **least-cost path** between a source and a destination.

- A **path** is a sequence of nodes $(x_1, x_2, \dots, x_p)$ such that each adjacent pair is connected by an edge.
- The **cost of a path** is the sum of all edge costs:
$$c(x_1, x_2) + c(x_2, x_3) + \dots + c(x_{p-1}, x_p)$$

- Between two nodes, many paths may exist, each with different costs. The algorithm must identify one or more paths with the **minimum total cost**.

Example: In the given graph (Figure 5.3), the least-cost path between source node **u** and destination node **w** is (u, x, y, w) with a total cost of **3**. If all edges in the graph have the same cost, then the **least-cost path is also the shortest path** (fewest number of links).

Classification of Routing Algorithms

When trying to find the least-cost path (e.g., from node **u** to **z** in Figure 5.3), people usually rely on a centralized thought process—looking at the whole network map, testing some possible paths, and picking one that seems minimal. This illustrates one important way to classify routing algorithms: **centralized vs. decentralized**.

1. Centralized vs. Decentralized

- **Centralized Routing Algorithm:**
 - Uses **complete, global knowledge** of the network (all nodes, all link costs).
 - The algorithm takes the full network map as input and computes the least-cost paths.
 - Can run in a single location (e.g., a central controller) or be replicated in each router.
 - Such algorithms are called **Link-State (LS) algorithms**, since they require knowledge of every link's state and cost.
 - Example: OSPF (Open Shortest Path First).
- **Decentralized Routing Algorithm:**
 - Each router starts with knowledge only of its **directly connected links**.
 - Routers iteratively exchange information with their **neighbors**, gradually refining their estimates of least-cost paths.
 - Each node maintains a **vector** of estimated costs to all other nodes.

- This is known as the **Distance-Vector (DV) algorithm**.
- Naturally suited to control planes where routers interact directly with each other.
- Example: RIP (Routing Information Protocol).

2. Static vs. Dynamic

- **Static Routing Algorithms:**

- Routes change very slowly, usually through **manual human intervention** (e.g., editing link costs in a configuration).
- Rarely used in large, evolving networks.

- **Dynamic Routing Algorithms:**

- Adjust routes automatically when **topology or traffic load changes**.
- Can update periodically or immediately when changes occur.
- Advantage: More responsive to failures and traffic shifts.
- Disadvantage: More prone to **routing loops** and **oscillations**.
- Example: Internet protocols like OSPF and BGP.

3. Load-Sensitive vs. Load-Insensitive

- **Load-Sensitive Algorithms:**

- Link costs reflect **current congestion levels**.
- Congested links get higher costs, causing routers to select alternative paths.
- Early ARPAnet routing was load-sensitive but faced major difficulties (e.g., instability).

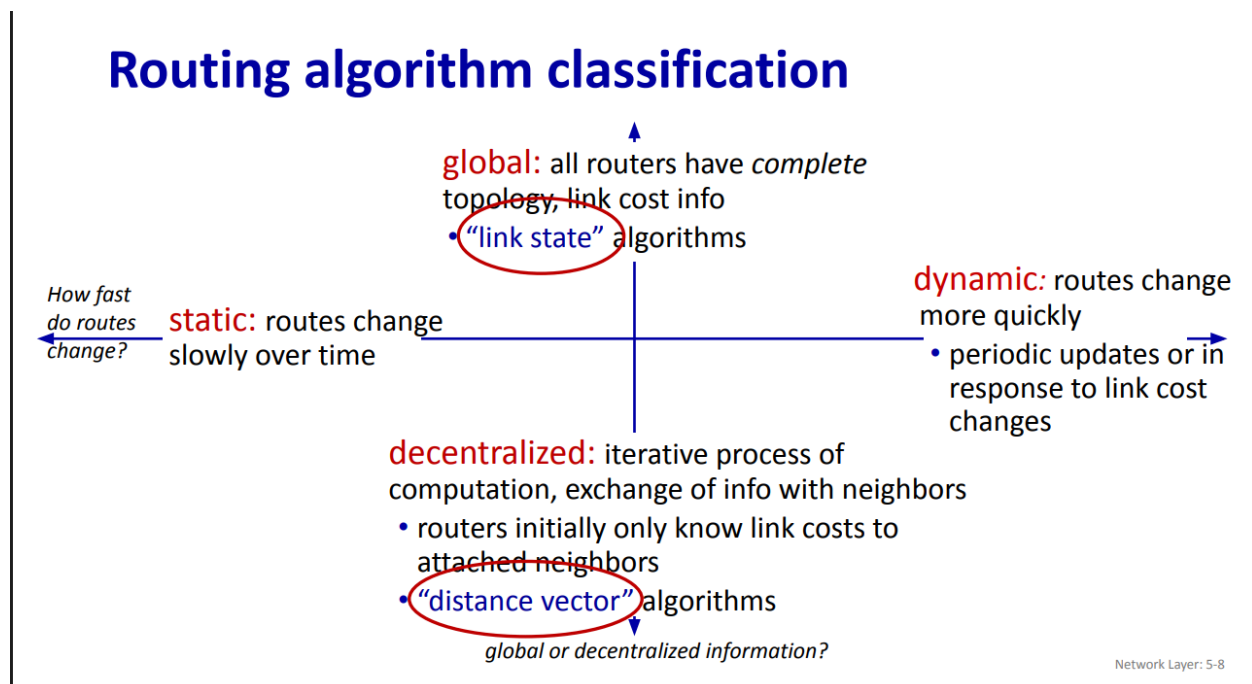
- **Load-Insensitive Algorithms:**

- Link costs remain fixed and do not explicitly reflect congestion.

- Today's Internet routing protocols (RIP, OSPF, BGP) fall into this category, as they do not adapt link cost based on traffic levels.

In summary, routing algorithms can be classified in three broad ways:

1. **Centralized (LS) vs. Decentralized (DV)** – global vs. distributed knowledge.
2. **Static vs. Dynamic** – fixed vs. adaptive to changes.
3. **Load-Sensitive vs. Load-Insensitive** – adaptive vs. fixed link costs relative to congestion.



Routing Algorithm Classification (Explained from the Diagram)

This diagram classifies **routing algorithms** using two main dimensions:

1. **How fast routes change?** (Horizontal axis)
2. **What kind of information is available?** (Vertical axis)

1. Static vs. Dynamic (Horizontal Axis)

- **Static Routing** (left side):
 - Routes change **very slowly** over time.
 - Typically requires **manual human intervention** to update routes.
 - Not adaptive to changes in network traffic or failures.
 - Example: Manually configured routes in small networks.
 - **Dynamic Routing** (right side):
 - Routes **adapt quickly** to network changes.
 - Updates can happen **periodically** or **immediately** in response to events (e.g., a link failure or cost change).
 - Common in large-scale, real-world networks.
 - Examples: OSPF, RIP, BGP.
-

2. Global vs. Decentralized (Vertical Axis)

- **Global (Top):**
 - Every router has **complete knowledge** of the network topology and link costs.
 - This is the **Link-State (LS) approach**.
 - Example: OSPF (Open Shortest Path First).
 - Process: Routers flood link-state information to all other routers → each router builds the same network map → runs **Dijkstra's algorithm** locally.
- **Decentralized (Bottom):**
 - Routers do **not** have full global knowledge.
 - Each router initially only knows the cost of links to its **direct neighbors**.

- Through iterative information exchange with neighbors, routers update their estimates of shortest paths.
 - This is the **Distance-Vector (DV) approach**.
 - Example: RIP (Routing Information Protocol).
-

3. Combining the Two Dimensions

- **Top-left (Global + Static):** Link-state with slow changes (rare in practice).
 - **Top-right (Global + Dynamic):** Standard **Link-State protocols** (e.g., OSPF) that update quickly when topology changes.
 - **Bottom-left (Decentralized + Static):** Rare case; would require manual updates with only local info.
 - **Bottom-right (Decentralized + Dynamic):** **Distance-Vector protocols** (e.g., RIP), where routers exchange updates iteratively with neighbors to adapt to changes.
-

✓ Key takeaway:

- **Link-State algorithms** = global knowledge.
- **Distance-Vector algorithms** = local knowledge + iterative updates.
- **Static vs. Dynamic** tells us how responsive the algorithm is to network changes.

Do you want me to also make a **table version** of this classification (easy for quick memorization for exams)?

Link-State Routing and Dijkstra's Algorithm

In a **Link-State (LS) algorithm**, the **entire network topology and all link costs** are known to each router. This is achieved through a **link-state broadcast**, where every node sends out link-state packets containing the identities and costs of its directly connected links. As a result, **all routers have an identical, complete map of the network**.

In the Internet, this is implemented by protocols like **OSPF (Open Shortest Path First)**, which use link-state broadcast techniques. Once every node has the same global view, each router can independently run the LS algorithm to compute the same set of least-cost paths.

The **most widely used LS algorithm** is **Dijkstra's Algorithm**,

Purpose of Dijkstra's Algorithm

- Computes the **least-cost path** from a single source node u to all other nodes in the network.
- It is **iterative**, and after the k -th iteration, the least-cost paths are known to k nodes.
- These k paths are guaranteed to have the **k smallest costs** among all paths in the network.

Notation Used in Dijkstra's Algorithm

- $D(v)$: Current cost of the least-cost path from the source node u to destination node v .
- $p(v)$: The previous node (neighbor) of v along the current least-cost path.
- N' : Set of nodes for which the least-cost path is definitively known.

Steps of the Algorithm (Centralized LS Approach)

Link-State (LS) Algorithm for Source Node u

```
1  Initialization:
2     $N' = \{u\}$ 
3    for all nodes  $v$ 
4      if  $v$  is a neighbor of  $u$ 
5        then  $D(v) = c(u, v)$ 
6      else  $D(v) = \infty$ 
7
8  Loop
9    find  $w$  not in  $N'$  such that  $D(w)$  is a minimum
10   add  $w$  to  $N'$ 
11   update  $D(v)$  for each neighbor  $v$  of  $w$  and not in  $N'$ :
12      $D(v) = \min(D(v), D(w) + c(w, v))$ 
13   /* new cost to  $v$  is either old cost to  $v$  or known
14      least path cost to  $w$  plus cost from  $w$  to  $v$  */
15 until  $N' = N$ 
```

▶ Network Routing: Dijkstra's Algorithm

This is the vdo link to understand

Autonomous Systems and Intra-AS Routing

Up to this point, routing algorithms were studied in a simplified model where the network was seen as just a collection of routers, all running the same routing algorithm and working in a uniform way. In practice, however, this model is **too simplistic** for two main reasons:

1. Scale

- The modern Internet has **hundreds of millions of routers**.
 - If every router stored information about all possible destinations, it would require **enormous memory**.
 - The overhead of **broadcasting connectivity and link cost updates** to every single router would be **extremely large**.
 - For example, a **distance-vector algorithm** with such a vast number of routers would **never converge** (too many iterations, too much delay).
 - Hence, routing must be broken down into smaller, manageable parts to reduce the **complexity of computation and communication overhead**.
-

2. Administrative Autonomy

- The Internet is a network of **ISPs**, and each ISP runs its own network of routers.
 - Each ISP usually wants to:
 - **Control its own network** freely (e.g., decide which routing algorithm to use internally).
 - **Hide details** of its internal network organization from outside networks.
 - This autonomy allows organizations to manage their networks independently while still remaining connected to the global Internet.
-

Solution: Autonomous Systems (ASs)

To address scale and autonomy, routers are organized into **Autonomous Systems (ASs)**:

- An **AS** is a group of routers under the **same administrative control**.
- Typically, all routers within an **ISP** and their internal links form one AS.
- Some ISPs use a **single, large AS** for their whole network, while others divide their network into **multiple ASs** connected together.

- Each AS is identified by a **globally unique Autonomous System Number (ASN)**, assigned by **ICANN regional registries** (similar to how IP addresses are assigned).
-

Routing within an AS

- Inside an AS, all routers:
 - Run the **same routing algorithm**.
 - Have complete information about each other.
 - The routing protocol used **inside** an AS is called an **Intra-AS Routing Protocol (Interior Gateway Protocol, IGP)**.
 - Examples include:
 - **RIP** (Routing Information Protocol)
 - **OSPF** (Open Shortest Path First)
 - **IS-IS** (Intermediate System to Intermediate System)
-

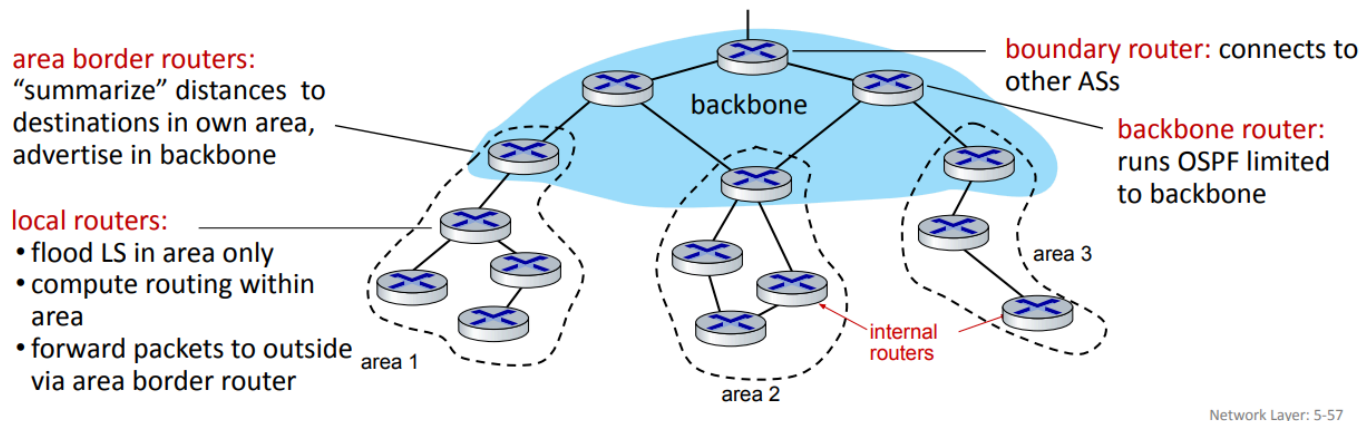
Key takeaway:

To handle the **scale** of the Internet and allow **administrative independence**, the global Internet is divided into **Autonomous Systems (ASs)**, each running its own **intra-AS routing protocol** for internal routing.

OSPF (Open Shortest Path First) Routing Protocol

Hierarchical OSPF

- **two-level hierarchy:** local area, backbone.
 - link-state advertisements flooded only in area, or backbone
 - each node has detailed area topology; only knows direction to reach other destinations



Network Layer: 5-57

Hierarchical OSPF (Open Shortest Path First)

This diagram explains how OSPF uses a **two-level hierarchical structure** to scale routing within large Autonomous Systems (AS). Instead of flooding routing information everywhere, OSPF divides the AS into **areas** and connects them through a special **backbone area**.

1. Two-Level Hierarchy

- **Local Areas:** Each area is a collection of routers that only exchange link-state information within the area.
- **Backbone Area (Area 0):** A special central area that connects all local areas together.

This reduces overhead: instead of every router knowing the full network topology, they only know their own area's details and how to reach other areas through the backbone.

2. Types of Routers in Hierarchical OSPF

- **Local Routers (Internal Routers):**
 - Operate **inside one area only**.
 - Flood LSAs (link-state advertisements) only within their area.
 - Compute routes within the area.
 - For destinations outside the area, forward packets to an **Area Border Router (ABR)**.
- **Area Border Routers (ABRs):**
 - Connect one area to the **backbone area**.
 - Summarize routes from their local area (instead of sending full details).
 - Advertise summarized routing info into the backbone.
 - Act as gateways for traffic leaving or entering their area.
- **Backbone Routers:**
 - Belong to the **backbone area**.
 - Run OSPF limited to the backbone.
 - Carry routing info between ABRs, ensuring inter-area connectivity.
- **Boundary Routers (AS Boundary Routers):**
 - Connect the OSPF Autonomous System to **other Autonomous Systems** (e.g., via BGP).
 - Handle external routes entering or leaving the AS.

3. How Routing Works in Hierarchical OSPF

- Inside an area:
 - Each router has **complete knowledge** of that area's topology.
 - Routing is computed using standard OSPF link-state flooding + Dijkstra.
- Between areas:
 - Routers don't know other areas' full topology.
 - They only know how to **reach destinations in other areas** via ABRs.
 - ABRs advertise summarized routes into the backbone, reducing overhead.
- The **backbone** is responsible for carrying inter-area traffic.

✓ Key takeaway:

Hierarchical OSPF divides the AS into **local areas** connected by a **backbone area**. Local routers manage routing inside their area, ABRs connect areas to the backbone, backbone routers carry inter-area traffic, and boundary routers connect to external ASs. This structure makes OSPF scalable, efficient, and manageable for large networks.

Would you like me to also make a **flow-style diagram (step sequence)** of how a packet is routed in hierarchical OSPF (from an internal router in Area 1 to a destination in Area 3)?

1. Basic Features of OSPF

- **Open Standard:** "Open" means the specification is publicly available (unlike Cisco's EIGRP, which was proprietary for many years).
- **Defined in RFC 2328:** OSPF version 2 is the most widely implemented version.
- **Link-State Protocol:**

- Routers flood **link-state advertisements (LSAs)** throughout the AS.
 - Each router builds an identical **complete network topology map**.
 - Each router then runs **Dijkstra's algorithm** locally to compute shortest-path trees, with itself as the root.
 - **Link Costs:**
 - Set by the **network administrator**.
 - Can be all equal (=1) → gives **minimum-hop routing**.
 - Can be **inversely proportional to capacity** → discourages traffic on low-bandwidth links.
 - OSPF does not enforce how costs should be set; it just provides the mechanisms.
-

2. Traffic Engineering with OSPF Link Weights

- Normally: **Link weights** → **Dijkstra's algorithm** → **Routing paths**.
 - In practice: Operators may **set link weights strategically** to achieve goals (e.g., balance traffic, minimize congestion).
 - Example: If operators know traffic patterns, they adjust link weights so OSPF distributes flows to minimize maximum link utilization.
 - Thus, link weights are sometimes chosen to **force desired routing outcomes**.
-

3. Flooding and Updates

- Routers broadcast LSAs to **all routers in the AS**, not just neighbors.
- Broadcast occurs:
 - Whenever a link's state changes (up/down or cost change).

- **Periodically (every 30 minutes)**, even if no change occurs (for robustness).
 - OSPF messages are carried directly over **IP (protocol number 89)**.
 - OSPF implements its own mechanisms for:
 - **Reliable flooding** of LSAs.
 - **HELLO messages** to check neighbor liveness.
 - **Database synchronization** to ensure routers share the same topology map.
-

4. Advanced Features of OSPF

1. Security:

- OSPF exchanges can be authenticated to prevent malicious attacks.
- **Simple authentication**: Password in plaintext (weak).
- **MD5 authentication**: Stronger, uses shared secret keys + hash functions. Protects against replay attacks using sequence numbers.

2. Multiple Same-Cost Paths:

- OSPF supports **Equal-Cost Multi-Path (ECMP)** routing.
- Traffic can be spread across multiple paths of equal cost (instead of choosing one).

3. Multicast Support:

- **MOSPF (Multicast OSPF, RFC 1584)** extends OSPF to handle multicast routing.
- Uses the same link-state database, with additional LSAs for multicast.

4. Hierarchical Structure:

- OSPF supports dividing an AS into **areas** for scalability.

- Within an area: Each router floods LSAs to all others in that area.
 - **Area Border Routers (ABRs)** connect areas to the **backbone area**.
 - The **backbone area** (Area 0) is central and routes traffic between other areas.
 - Inter-area routing requires packets to travel:
 - From source → ABR → Backbone → Destination's ABR → Final destination.
-

5. Summary of OSPF's Strengths

- Open standard and widely supported.
 - Efficient use of **link-state flooding + Dijkstra's algorithm**.
 - Flexible link cost configuration (can be adapted for **traffic engineering**).
 - Built-in support for **security, ECMP, multicast, and hierarchy**.
 - Scalable and robust, making it the dominant intra-AS protocol today.
-

✓ **Key takeaway:** OSPF is not just a shortest-path protocol—it's a **sophisticated, secure, and hierarchical link-state protocol** that allows administrators to fine-tune routing behavior to meet performance and traffic-engineering goals.

Inter-AS Routing and the Role of BGP (Border Gateway Protocol)

So far, we studied **intra-AS routing protocols** (like OSPF and IS-IS), which determine how packets are routed **within a single AS**. However, the Internet is composed of **thousands of interconnected Autonomous Systems (ASs)**, so we also need a way to route packets

between ASs. This is the role of the **Border Gateway Protocol (BGP)**, the only inter-AS routing protocol used in today's Internet.

1. Why We Need BGP

- Intra-AS protocols only decide how to forward packets **inside an AS**.
 - For packets whose destination lies **outside the AS**, routers need a mechanism to learn how to reach external networks.
 - BGP provides this mechanism—it essentially **glues the Internet together** by allowing ASs to share reachability information.
-

2. What BGP Routes To

- Unlike intra-AS protocols that compute routes to individual IP addresses, BGP routes to **CIDRized prefixes**.
 - A **prefix** represents a block of IP addresses (subnet). Example:
 - Prefix **138.16.68/22** covers **1,024 IP addresses**.
 - In forwarding tables, entries look like:
(x,I)(x, I)
where **x** = prefix (destination subnet) and **I** = outgoing interface.
-

3. BGP's Responsibilities

BGP has two fundamental jobs:

1. Prefix Advertisement (Reachability Information):

- Each subnet can announce itself to the rest of the Internet by advertising its prefix.
- BGP ensures this information propagates globally.

- Without BGP, subnets would remain isolated, invisible, and unreachable.

2. Best-Route Selection:

- A router may learn multiple possible routes to the same prefix from different neighbors.
 - BGP defines a **route selection procedure** to choose the **best path**.
 - Importantly, “best” is not only about cost or hop count—it also includes **policy decisions** (business agreements, peering rules, traffic preferences, etc.).
-

4. Nature of BGP

- BGP is a **decentralized** and **asynchronous** protocol.
 - Conceptually, it is similar to a **distance-vector protocol**, but extended to handle Internet-scale routing with policies.
 - BGP is considered one of the **most critical Internet protocols**—second in importance only to IP itself—since without it, ISPs and ASs could not interconnect in a global Internet.
-

Key takeaway:

- **Intra-AS protocols** (like OSPF, IS-IS) handle routing *within* an AS.
- **BGP** handles routing *between* ASs, providing global reachability by advertising prefixes and selecting best routes based on **reachability + policy**.

Would you like me to now **explain the BGP route advertisement and selection process step by step** (how updates flow between ASs and how a router chooses its final route)?

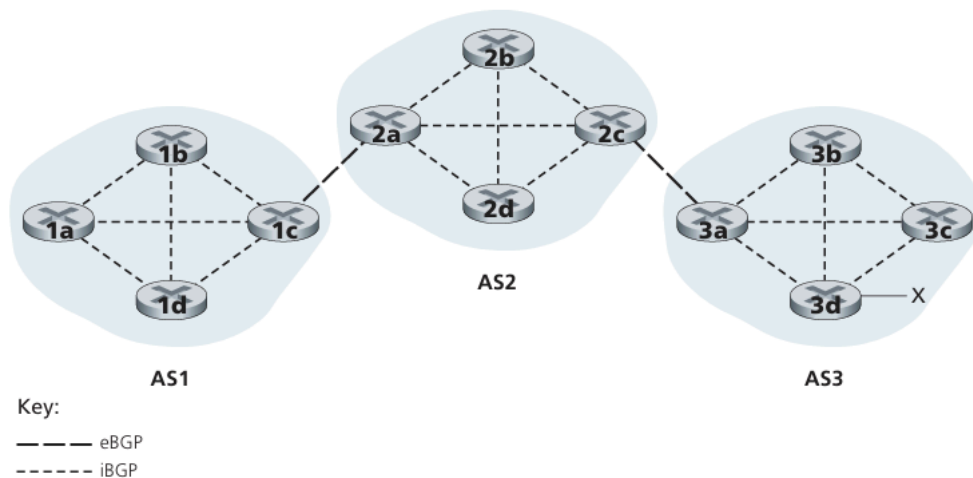


Figure 5.9 ♦ eBGP and iBGP connections

Advertising BGP Route Information

The figures show how **BGP (Border Gateway Protocol)** advertises reachability information across multiple **Autonomous Systems (ASs)**. The goal is to let all routers in the Internet know how to reach a destination prefix (subnet).

1. Routers and Their Roles

- **Internal routers:** Connect only to hosts/routers inside the same AS.
 - Example: Routers 1a, 1b, 1d in AS1.
- **Gateway routers:** Connect to routers in **other ASs**.
 - Example: Router 1c (AS1), 2a & 2c (AS2), 3a (AS3).

2. Scenario

- Prefix **x** exists inside **AS3** (subnet attached to router 3d).

- The goal: advertise reachability to **x** to **AS2** and **AS1**, so that all routers in all three ASs know how to reach x.
-

3. How BGP Advertises Routes

1. **AS3** → **AS2**:

- Gateway router **3a** (AS3) advertises prefix x to **2c** (AS2) using **external BGP (eBGP)**.
- Message looks like: “**AS3 x**” (prefix x reachable via AS3).

2. **Inside AS2**:

- Router **2c** then propagates this information to all routers in AS2 using **internal BGP (iBGP)**.
- Message: “**AS3 x**”.
- Now every router in AS2 knows that x is reachable via AS3.

3. **AS2** → **AS1**:

- Gateway router **2a** then advertises to gateway router **1c** (in AS1) via **eBGP**.
- Message: “**AS2 AS3 x**” (meaning: go through AS2, then AS3, to reach x).

4. **Inside AS1**:

- Router **1c** distributes this information to all other routers in AS1 using **iBGP**.
- Message: “**AS2 AS3 x**”.

✓ Now, all routers in **AS1** and **AS2** know that prefix x exists and know the **AS path** to reach it.

4. BGP Connections

- **eBGP (external BGP)**: Between routers in **different ASs**.

- Example: $1c \leftrightarrow 2a$, $2c \leftrightarrow 3a$.
 - **iBGP (internal BGP)**: Between routers **inside the same AS**.
 - Typically, a **full mesh** of iBGP sessions is set up so all routers in the AS can share reachability information.
 - Note: iBGP connections are logical TCP sessions (port 179), not necessarily physical links.
-

5. Multiple Paths

- In the simple case, only one path exists: **AS2** → **AS3** → **x**.
 - But in a more complex network (like in Figure 5.10 with an extra link from **1d** to **3d**), there can be **multiple paths**:
 - Path 1: **AS2** → **AS3** → **x** (via 1c).
 - Path 2: **AS3** → **x** (direct via 1d).
 - In such cases, BGP's **route selection rules** determine which path becomes the **best route** (based on policy, AS path length, and other factors).
-

✓ Key takeaway:

BGP spreads reachability information by using **eBGP across AS boundaries** and **iBGP within an AS**. Each AS learns not just the existence of a prefix (like x), but also the **AS path** to get there. This mechanism allows the Internet's thousands of ASs to interconnect seamlessly.

Determining the Best Routes in BGP

Routers often learn **multiple routes** to the same destination subnet. The challenge is deciding **which route is the best** and should be placed in the forwarding table. BGP does this using attributes and a **route-selection algorithm**.

1. BGP Attributes

When a router advertises a prefix, it includes additional information called **BGP attributes**. A prefix plus its attributes = a **BGP route**.

Two key attributes:

- **AS-PATH:**
 - List of ASs that the advertisement has traversed.
 - Example: In Figure 5.10, prefix x may be advertised as:
 - **AS2 AS3 x** (via AS2), or
 - **AS3 x** (directly via AS3).
 - Used both for route selection (shorter AS-PATH is better) and loop prevention (if a router sees its own ASN in the list, it rejects the route).
- **NEXT-HOP:**
 - IP address of the router interface that begins the AS-PATH.
 - Example:
 - Route “AS2 AS3 x” has NEXT-HOP = router 2a’s interface.
 - Route “AS3 x” has NEXT-HOP = router 3d’s interface.

So, in Figure 5.10, each router in AS1 knows two possible routes to prefix x:

1. NEXT-HOP = 2a, AS-PATH = (AS2, AS3), prefix = x

2. NEXT-HOP = 3d, AS-PATH = (AS3), prefix = x
-

2. Hot Potato Routing

- In **hot potato routing**, the router chooses the NEXT-HOP that is **closest** (lowest intra-AS cost).
- Example: Router 1b compares:
 - Cost to NEXT-HOP 2a (via AS2).
 - Cost to NEXT-HOP 3d (via AS3).
- Whichever path has the smaller internal cost is chosen.
- Idea: Push the packet **out of your AS as fast as possible**—like getting rid of a hot potato.

Steps (Figure 5.11):

1. Learn from BGP that prefix x is reachable via multiple gateways.
2. Use intra-AS routing (e.g., OSPF) to compute cost to each gateway.
3. Choose the gateway with the smallest cost (**hot potato choice**).
4. Update forwarding table with entry (x, I), where I = interface to chosen gateway.

Note: This approach is **selfish**—it minimizes cost inside the AS but ignores cost in other ASs.

3. BGP Route-Selection Algorithm

In practice, BGP doesn't rely only on hot potato routing. It applies a **sequence of rules** to pick the best route:

1. **Local Preference:**

- Each route is given a **local preference** value.
- Higher preference = better route.
- Policy-based decision (set by administrators).

2. Shortest AS-PATH:

- If multiple routes remain, choose the one with the **fewest AS hops**.

3. Hot Potato Routing:

- If still tied, choose the route with the **closest NEXT-HOP**.

4. Tie-breakers:

- If multiple routes remain, break ties using **BGP router identifiers**.

4. Example: Router 1b in Figure 5.10

- Route 1: AS-PATH = AS2 → AS3 → x
- Route 2: AS-PATH = AS3 → x

If hot potato alone was used, router 1b might choose via AS2 (shorter internal path).
But according to BGP's full algorithm:

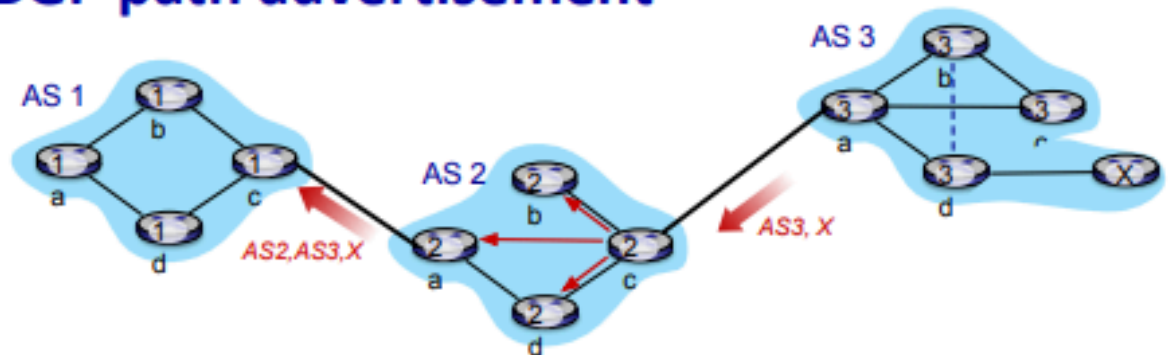
- Rule 1 (local pref): assume equal.
- Rule 2 (AS-PATH): AS3-only path is shorter → chosen.
So BGP prefers the **AS3 direct path** over the AS2 detour.

✓ Key takeaway:

BGP combines **policy (local pref)**, **efficiency (short AS-PATH)**, and **self-interest (hot potato routing)** to decide the best route. Unlike intra-AS protocols, BGP is not just about shortest paths—it balances **policy and performance**.

Do you want me to also make a **step-by-step worked example** showing exactly how router 1b updates its forwarding table with both routes and then applies the rules?

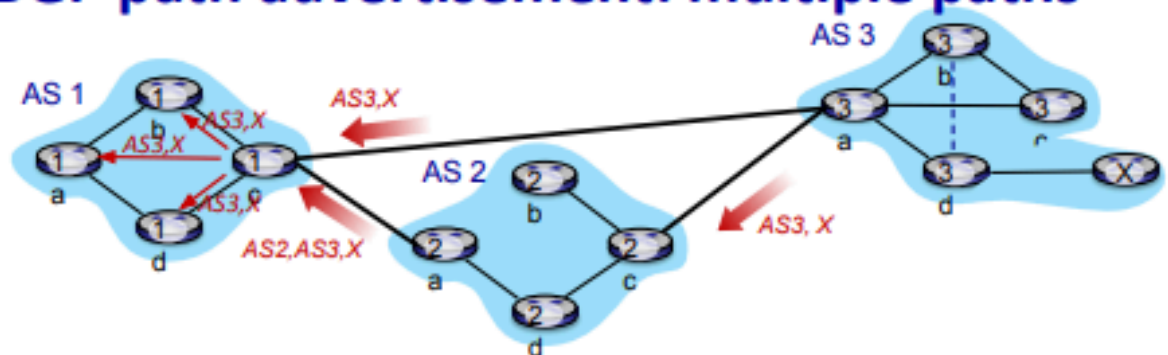
BGP path advertisement



- AS2 router 2c receives path advertisement **AS3,X** (via eBGP) from AS3 router 3a
- based on AS2 policy, AS2 router 2c accepts path AS3,X, propagates (via iBGP) to all AS2 routers
- based on AS2 policy, AS2 router 2a advertises (via eBGP) path **AS2, AS3, X** to AS1 router 1c

Network Layer: 5-65

BGP path advertisement: multiple paths

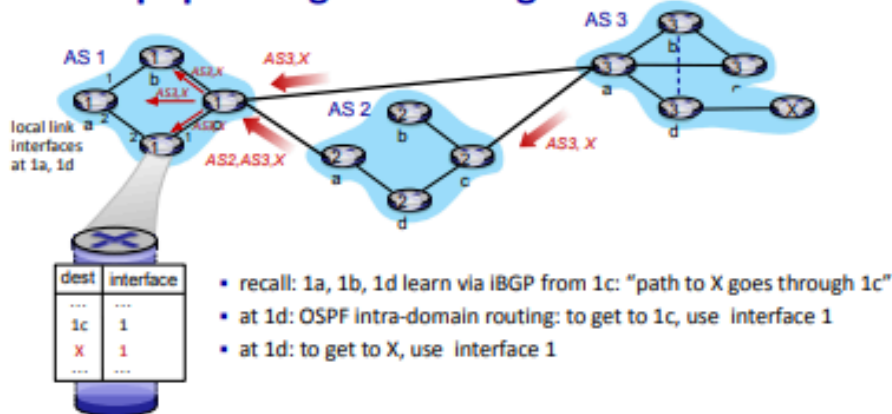


gateway router may learn about **multiple** paths to destination:

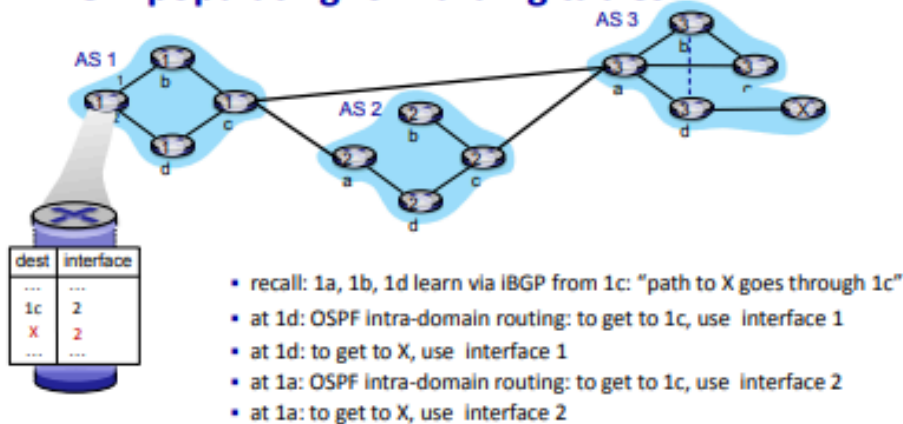
- AS1 gateway router 1c learns path **AS2,AS3,X** from 2a
- AS1 gateway router 1c learns path **AS3,X** from 3a
- based on *policy*, AS1 gateway router 1c chooses path **AS3,X** and advertises path within AS1 via iBGP

Network Layer: 5-66

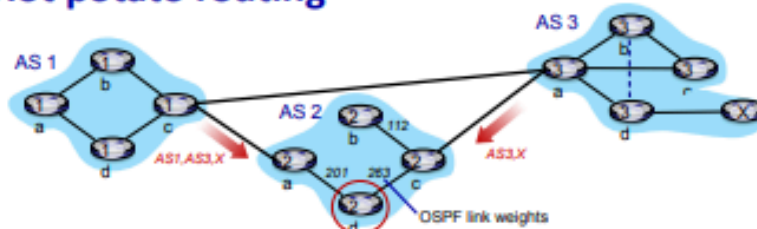
BGP: populating forwarding tables



BGP: populating forwarding tables

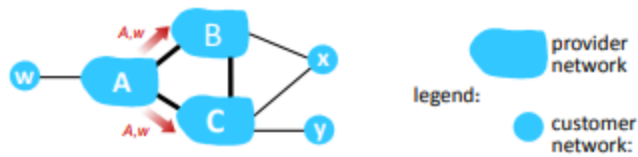


Hot potato routing



- 2d learns (via iBGP) it can route to X via 2a or 2c
- hot potato routing**: choose local gateway that has least *intra-domain* cost (e.g., 2d chooses 2a, even though more AS hops to X): don't worry about inter-domain cost!

BGP: achieving policy via advertisements



ISP only wants to route traffic to/from its customer networks (does not want to carry transit traffic between other ISPs – a typical “real world” policy)

- A advertises path Aw to B and to C
- B *chooses not to advertise* BA_w to C!
 - B gets no “revenue” for routing CBA_w, since none of C, A, w are B’s customers
 - C does *not* learn about CBA_w path
- C will route CA_w (not using B) to get to w

Network Layer: 5-70

BGP: achieving policy via advertisements (more)



ISP only wants to route traffic to/from its customer networks (does not want to carry transit traffic between other ISPs – a typical “real world” policy)

- A,B,C are **provider networks**
- x,w,y are **customer** (of provider networks)
- x is **dual-homed**: attached to two networks
- **policy to enforce**: x does not want to route from B to C via x
 - .. so x will not advertise to B a route to C

BGP:

Achieving Policy via Advertisements

These slides show how **BGP (Border Gateway Protocol)** enforces **routing policies** by controlling **what advertisements are sent** between Autonomous Systems (ASs). Unlike intra-AS routing, BGP routing decisions are heavily influenced by **business and economic relationships** (provider ↔ customer, peer ↔ peer).

1. Scenario (Top Slide)

- **A, B, C** = Provider networks (ISPs).
- **w, x, y** = Customer networks (attached to providers).
- **Policy:** An ISP only wants to route traffic **to/from its customer networks**, but **not act as transit** for other ISPs.

How it plays out:

1. **A advertises path Aw** to both B and C.
 - This tells B and C that prefix w is reachable via A.
2. **B chooses not to advertise BAw to C.**
 - Why? Because then traffic from C to w would pass through B, making B act as a **transit provider** without revenue.
3. **Effect of this policy:**
 - B does not forward C's traffic to A/w.
 - C does not learn about the path C–B–A–w.
 - Instead, C learns directly from A (path C–A–w).
 - B avoids “free work” (carrying transit traffic without payment).

✓ **Key idea:** BGP allows providers to **filter route advertisements** to enforce economic policies.

2. Scenario with Dual-Homing (Bottom Slide)

- **x** = Customer network, but it is **dual-homed** (connected to both A and B).
- A, B, C are still providers; x, y are customers.

Policy enforcement:

- x may prefer to route traffic in certain ways, but B **does not want traffic to pass through B to reach x via C**.
- So B will **not advertise to C the route to x through B**.

✓ This enforces the policy:

- Providers carry **traffic for their customers** (x, y, w).
 - Providers **do not act as free transit carriers** for other providers' traffic.
-

3. General Real-World BGP Policy

- **Customer routes are always advertised** (providers make money carrying customer traffic).
 - **Peer/provider routes are often filtered** (to avoid becoming unpaid transit).
 - BGP's flexibility in **advertising or withholding routes** is the mechanism by which **business agreements shape Internet routing**.
-

✓ **Key takeaway:**

BGP is not just about finding shortest paths—it is a **policy-driven protocol**. ISPs use BGP advertisements (or suppression of advertisements) to enforce **economic relationships**:

- Carry traffic for **your customers**.
- **Do not carry free transit** for competitors.

ICMP: Internet Control Message Protocol

ICMP (defined in **RFC 792**) is a protocol used by hosts and routers to communicate **network-layer information**. It is mainly used for **error reporting** and **diagnostics**.

1. Position in Protocol Stack

- ICMP is often considered part of IP.
 - Architecturally, it lies **just above IP**:
 - ICMP messages are carried **inside IP datagrams** (IP protocol number = 1).
 - Like TCP/UDP, ICMP is treated as IP payload.
 - When a host receives an ICMP packet, it is demultiplexed to ICMP (similar to TCP or UDP).
-

2. ICMP Message Format

- Each ICMP message has:
 - A **type field** (indicates message category).
 - A **code field** (gives more detail about the type).
 - The **IP header + first 8 bytes of the datagram that caused the error** (so the sender knows which packet failed).
-

3. Common ICMP Message Types (Figure 5.19)

Type	Code	Description
0	0	Echo reply (used in Ping response)

3	0	Destination network unreachable
3	1	Destination host unreachable
3	2	Destination protocol unreachable
3	3	Destination port unreachable
3	6	Destination network unknown
3	7	Destination host unknown
4	0	Source quench (congestion control, obsolete)
8	0	Echo request (Ping request)
9	0	Router advertisement
10	0	Router discovery
11	0	TTL expired
12	0	IP header bad

4. Typical Uses of ICMP

1. Error Reporting

- Example: “Destination network unreachable” comes from ICMP when a router cannot find a path to the host.

2. Ping (Echo Request/Reply)

- **Type 8 Code 0**: Echo request.
- **Type 0 Code 0**: Echo reply.
- Used to test reachability of a host.

3. Source Quench (Obsolete)

- Sent by routers to ask a sender to slow down (for congestion control).
- Rarely used today because TCP has its own congestion control.

4. Traceroute

- Implemented using ICMP:
 - Source sends UDP packets with increasing TTL values (1, 2, 3, ...).
 - Each router decrements TTL. When TTL = 0, router discards packet and sends back **ICMP Type 11 Code 0 (TTL expired)**.
 - The source records each router’s address + round-trip time.
 - Eventually, the packet reaches the destination, which responds with **ICMP Type 3 Code 3 (Port unreachable)**, signaling the end.
- This allows the source to learn the **path of routers** between source and destination.

5. ICMP in IPv6 (ICMPv6)

- Defined in **RFC 4443**.
 - Reorganizes type/code values.
 - Adds new messages for IPv6 features:
 - “Packet Too Big.”
 - “Unrecognized IPv6 option.”
 - Plays a more central role in IPv6 (e.g., Neighbor Discovery).
-

 Key takeaway:

ICMP is essential for error reporting, diagnostics (ping, traceroute), and network management. While not used for actual data transfer, it provides critical control-plane communication that makes IP networking robust and manageable.

Do you want me to also create a **flowchart showing how ICMP is used in Ping and Traceroute** (step by step)?